Python - Guida Completa a Pandas e Visualizzazione

Introduzione

Questa guida raccoglie e organizza in modo coerente tutti i concetti teorici e pratici trattati nei materiali Python del repository. Il percorso copre due aree fondamentali dell'analisi dati:

1. **Pandas**: La libreria principale per la manipolazione e l'analisi dei dati in Python
2. **Visualizzazione**: Tecniche di rappresentazione grafica con matplotlib e seaborn

L'approccio didattico segue una progressione logica: dai concetti fondamentali di Series e DataFrame fino alle operazioni avanzate di aggregazione, gestione del tempo, e creazione di visualizzazioni efficaci. Ogni sezione include spiegazioni teoriche, esempi pratici, e best practice per lo sviluppo di codice pulito ed efficiente.

Parte I: Fondamenti di Pandas

1.1 Cos'è Pandas

Pandas è una libreria Python open-source progettata per la manipolazione e l'analisi dei dati. Fornisce strutture dati ad alte prestazioni e strumenti per lavorare con dataset strutturati, rendendo l'analisi dati veloce, facile e flessibile.

Concetti chiave:

- **Series**: Una struttura dati unidimensionale etichettata, simile a una colonna di un database o a una serie temporale
- **DataFrame**: Una struttura dati bidimensionale etichettata, simile a un foglio di calcolo o tabella SQL, con colonne di potenziali tipi diversi

Perché Pandas è essenziale:

- Operazioni vettorializzate (veloci senza loop espliciti)
- Gestione integrata dei dati mancanti
- Strumenti potenti per aggregazione e raggruppamento

- Integrazione con altre librerie (NumPy, matplotlib, scikit-learn)

1.2 Creazione di Series e DataFrame

Creazione di una Series:

```
import pandas as pd
import numpy as np

# Da lista
s = pd.Series([10, 20, 30], name="punti")

# Da dizionario
s = pd.Series({'a': 10, 'b': 20, 'c': 30})
```

Creazione di un DataFrame:

```
# Da dizionario di liste
data = {
    'nome': ['Anna', 'Luca', 'Marco', 'Sara'],
    'eta': [25, 40, 30, 22],
    'citta': ['Milano', 'Roma', 'Napoli', 'Milano'],
    'spesa': [50.5, 120.0, 80.0, 30.0]
}
df = pd.DataFrame(data)

# Da lista di dizionari
df = pd.DataFrame([
    {'nome': 'Anna', 'eta': 25},
    {'nome': 'Luca', 'eta': 40}
])
```

Visualizzazione:

```
# Prime righe
df.head()        # Prime 5 righe
df.head(10)      # Prime 10 righe

# Ultime righe
df.tail()        # Ultime 5 righe

# Informazioni generali
df.info()        # Tipi di dato, missing values, memoria

# Statistiche descrittive
df.describe()    # Media, std, min, max, quartili per colonne numeriche

# Forma del DataFrame
df.shape         # (righe, colonne)

# Colonne
df.columns       # Indice delle colonne
```

1.3 Lettura e Scrittura di File

Formati supportati:

```
# CSV
df = pd.read_csv('file.csv')
df.to_csv('output.csv', index=False)  # index=False evita di salvare l'indice

# Excel
df = pd.read_excel('file.xlsx')
df.to_excel('output.xlsx', index=False, sheet_name='Dati')

# JSON
df = pd.read_json('file.json')
df.to_json('output.json', orient='records')

# SQL (richiede connessione)
df = pd.read_sql_query("SELECT * FROM tabella", connection)
df.to_sql('nome_tabella', connection, if_exists='replace', index=False)

# Parquet (formato binario efficiente)
df = pd.read_parquet('file.parquet')
df.to_parquet('output.parquet')
```

Parametri comuni di `read_csv`:

```
df = pd.read_csv(
    'file.csv',
    sep=',',              # Delimitatore (default: comma)
    header=0,             # Righe da usare come nomi colonne
    names=['a', 'b'],     # Nomi colonne personalizzati
    index_col=0,          # Usare una colonna come indice
    usecols=['a', 'b'],   # Caricare solo colonne specifiche
    dtype={'eta': int},   # Tipi di dato specifici
    parse_dates=['data'], # Convertire colonne in datetime
    na_values=['NA', ''], # Valori da considerare come NaN
    skiprows=1,           # Saltare righe iniziali
    encoding='utf-8'      # Encoding del file
)
```

Parte II: Selezione e Indicizzazione

2.1 Selezione di Colonne

```
# Singola colonna (ritorna una Series)
df['nome']

# Multiple colonne (ritorna un DataFrame)
df[['nome', 'eta']]

# Con attributo (solo se il nome è valido identificatore)
df.nome
```

2.2 Selezione di Righe: loc vs iloc

`loc` **(label-based):** Seleziona per etichetta/nome

```
# Riga specifica per indice
df.loc[0]           # Prima riga (indice 0)
df.loc[2]           # Terza riga (indice 2)

# Intervallo di righe (inclusivo agli estremi)
df.loc[0:3]         # Righe da 0 a 3 (INCLUDE 3!)

# Righe specifiche
df.loc[[0, 2, 4]]   # Righe 0, 2 e 4

# Colonne specifiche insieme alle righe
df.loc[0:3, ['nome', 'eta']]

# Condizione booleana
df.loc[df['eta'] > 25]
```

`iloc` (position-based): Seleziona per posizione numerica

```
# Riga specifica per posizione
df.iloc[0]          # Prima riga
df.iloc[2]          # Terza riga

# Intervallo di righe (ESCLUSIVO all'estremo finale)
df.iloc[0:3]        # Righe 0, 1, 2 (NON include 3!)

# Righe specifiche
df.iloc[[0, 2, 4]]

# Slicing di righe e colonne
df.iloc[0:3, 0:2]   # Prime 3 righe, prime 2 colonne

# Tutto
df.iloc[:]          # Tutto il DataFrame
df.iloc[:, 0]       # Prima colonna (tutte le righe)
df.iloc[0, :]       # Prima riga (tutte le colonne)
```

Regola fondamentale:

- **`loc`** : Inclusivo (`0:3` include 0, 1, 2, 3)
- **`iloc`** : Esclusivo (`0:3` include 0, 1, 2, NON include 3)

2.3 Query e Maschere Booleane

```
# Filtro semplice
df[df['eta'] > 25]

# Condizione combinata con AND (&)
df[(df['citta'] == 'Milano') & (df['eta'] >= 30)]

# Condizione combinata con OR (|)
df[(df['citta'] == 'Milano') | (df['citta'] == 'Roma')]

# NOT con ~
df[~(df['eta'] > 30)]  # Età <= 30

# isin (appartiene a)
df[df['citta'].isin(['Milano', 'Roma'])]

# between (intervallo)
df[df['eta'].between(20, 40)]  # 20 <= eta <= 40

# query (sintassi più leggibile)
df.query("eta > 25 and citta == 'Milano'")
df.query("eta between 20 and 40")
```

Importante: Quando si combinano condizioni, ogni condizione deve essere tra parentesi:

```
# CORRETTO
df[(df['citta'] == 'Milano') & (df['eta'] > 25)]

# SBAGLIATO (genera errore)
df[df['citta'] == 'Milano' & df['eta'] > 25]
```

Parte III: Manipolazione dei Dati

3.1 Aggiunta e Modifica di Colonne

```python
# Nuova colonna da operazione vettoriale
df['spesa_iva'] = df['spesa'] * 1.22  # Aggiunge 22% IVA

# Nuova colonna da condizione (booleana)
df['maggiorenne'] = df['eta'] >= 18

# Nuova colonna da funzione applicata riga per riga
def classifica_eta(eta):
    if eta <= 18:
        return "Giovane"
    elif eta <= 65:
        return "Adulto"
    else:
        return "Senior"

df['categoria_eta'] = df['eta'].apply(classifica_eta)

# Nuova colonna da lambda
df['eta_raddoppiata'] = df['eta'].apply(lambda x: x * 2)

# Modifica colonna esistente
df['eta'] = df['eta'] + 1

# Rimozione colonna
df = df.drop('spesa_iva', axis=1)  # axis=1 significa colonne
# oppure
del df['spesa_iva']
```

Operazioni vettorializzate: Le operazioni su colonne DataFrame sono automaticamente vettorializzate, quindi sono molto più veloci di loop Python espliciti.

3.2 Gestione dei Missing Values (NaN)

I **missing values** (valori mancanti) sono rappresentati come `NaN` (Not a Number) in Pandas.

Rilevamento:

```python
# Quali valori sono NaN?
df.isna()          # DataFrame booleano
df.isnull()        # Stesso di isna()

# Conteggio NaN per colonna
df.isna().sum()

# Percentuale NaN per colonna
df.isna().mean() * 100

# Righe con almeno un NaN
df[df.isna().any(axis=1)]
```

Strategie di gestione:

1. Rimozione (dropna):

```python
# Rimuovi righe con almeno un NaN
df_clean = df.dropna()

# Rimuovi righe solo se tutte le colonne specificate sono NaN
df_clean = df.dropna(subset=['eta', 'spesa'])

# Rimuovi colonne con almeno un NaN (raro)
df_clean = df.dropna(axis=1)

# Rimuovi solo se tutte le colonne sono NaN
df_clean = df.dropna(how='all')
```

2. Imputazione (fillna):

```python
# Sostituisci con valore costante
df['colonna'] = df['colonna'].fillna(0)

# Sostituisci con media
df['eta'] = df['eta'].fillna(df['eta'].mean())

# Sostituisci con mediana (più robusta agli outlier)
df['spesa'] = df['spesa'].fillna(df['spesa'].median())

# Sostituisci con moda (per categoriche)
df['citta'] = df['citta'].fillna(df['citta'].mode()[0])

# Sostituisci con valore forward (ultimo valore valido)
df.fillna(method='ffill')

# Sostituisci con valore backward (prossimo valore valido)
df.fillna(method='bfill')

# Sostituisci con valore interpolato (per serie temporali)
df['colonna'] = df['colonna'].interpolate()
```

3. Gestione avanzata:

```python
# Sostituire con "Unknown" per categorie
df['citta'] = df['citta'].cat.add_categories(["Unknown"])
df['citta'] = df['citta'].fillna("Unknown")

# Usare imputazione multivariata (scikit-learn)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
df[['eta', 'spesa']] = imputer.fit_transform(df[['eta', 'spesa']])
```

Best Practice per Missing Values:

Percentuale Missing	Strategia Consigliata
< 5%	Rimuovi le righe con missing
5-15%	Imputa con media/mediana/moda
15-30%	Considera imputazione avanzata o crea flag
> 30%	Valuta se eliminare la colonna

3.3 Rimozione dei Duplicati

```
# Identifica righe duplicate
df.duplicated()          # Series booleana
df.duplicated().sum()    # Conteggio duplicati

# Rimuovi duplicati (considera tutte le colonne)
df_clean = df.drop_duplicates()

# Rimuovi duplicati basati su colonne specifiche
df_clean = df.drop_duplicates(subset=['nome', 'citta'])

# Mantenere prima o ultima occorrenza
df_clean = df.drop_duplicates(keep='first')   # Default
df_clean = df.drop_duplicates(keep='last')    # Mantiene l'ultima
df_clean = df.drop_duplicates(keep=False)     # Rimuove tutte le occorrenze duplicate
```

3.4 Clonazione di DataFrame

```
# Copia superficiale (riferimento parziale)
df_shallow = df

# Copia profonda (oggetto indipendente)
df_deep = df.copy()
df_deep = df.copy(deep=True)

# Importante: sempre usare .copy() quando si crea un DataFrame derivato
# Altrimenti si può verificared il "SettingWithCopyWarning"
```

Parte IV: Aggregazione e Raggruppamento

4.1 Il Concetto di GroupBy

Il metodo `groupby()` divide i dati in gruppi basati su una o più colonne, permettendo di applicare operazioni di aggregazione a ciascun gruppo separatamente.

Analogia: Pensate a `groupby` come al `GROUP BY` di SQL.

4.2 Operazioni di Aggregazione Base

```python
# Somma per gruppo
df.groupby('Prodotto')['Vendite'].sum()

# Media per gruppo
df.groupby('Prodotto')['Vendite'].mean()

# Conteggio per gruppo
df.groupby('Prodotto')['Vendite'].count()

# Min e Max
df.groupby('Prodotto')['Vendite'].agg(['min', 'max'])

# Statistiche multiple
df.groupby('Prodotto')['Vendite'].agg(['count', 'sum', 'mean', 'max', 'min'])

# Descrizione completa
df.groupby('Prodotto')['Vendite'].describe()
```

4.3 Aggregazione Multiple Colonne

```python
# Aggregazione su più colonne
df.groupby('Prodotto').agg({
    'Vendite': ['sum', 'mean'],
    'Quantita': 'sum',
    'Prezzo': 'mean'
})

# Aggregazione con nomi personalizzati (Pandas >= 0.25)
df.groupby('Prodotto').agg(
    totale_vendite=('Vendite', 'sum'),
    media_vendite=('Vendite', 'mean'),
    totale_quantita=('Quantita', 'sum')
)
```

4.4 GroupBy Multi-Chiave

```python
# Raggruppa per più colonne
df.groupby(['Prodotto', 'Citta'])['Vendite'].sum()

# Con aggregazioni multiple
df.groupby(['Prodotto', 'Citta']).agg(
    totale=('Vendite', 'sum'),
    media=('Vendite', 'mean'),
    conteggio=('Vendite', 'count')
)
```

4.5 Transform: Calcoli per Riga

Il metodo `transform()` è fondamentale perché restituisce un DataFrame/Series con la **stessa forma** dell'input, permettendo di creare nuove colonne basate su calcoli di gruppo.

```python
# Totale vendite per prodotto (aggiunto come nuova colonna)
df['totale_prodotto'] = df.groupby('Prodotto')['Vendite'].transform('sum')

# Quota sul totale del prodotto
df['quota_prodotto'] = df['Vendite'] / df.groupby('Prodotto')['Vendite'].transform('sum')

# Media per gruppo (per riga)
df['media_prodotto'] = df.groupby('Prodotto')['Vendite'].transform('mean')

# Differenza dalla media del gruppo
df['diff_media'] = df['Vendite'] - df.groupby('Prodotto')['Vendite'].transform('mean')

# Rank all'interno del gruppo
df['rank_prodotto'] = df.groupby('Prodotto')['Vendite'].rank(ascending=False, method='dense'
```

4.6 Funzioni di Aggregazione Disponibili

```python
# Statistiche descrittive
df.groupby('colonna').agg(['sum', 'mean', 'median', 'std', 'var'])

# Conteggi
df.groupby('colonna').agg(['count', 'nunique'])

# Min/Max
df.groupby('colonna').agg(['min', 'max'])

# Percentili
df.groupby('colonna')['colonna_valore'].agg(['q1', 'q3', 'median'])

# Custom functions
df.groupby('colonna')['valore'].agg(lambda x: x.max() - x.min())

# Multiple custom functions
df.groupby('colonna')['valore'].agg([
    ('range', lambda x: x.max() - x.min()),
    ('coef_var', lambda x: x.std() / x.mean())
])
```

Parte V: Tabelle Pivot

5.1 Differenza tra pivot e pivot_table

`pivot()` : Richiede chiavi uniche (indice e colonne devono identificare univocamente ogni riga)

`pivot_table()` : Gestisce duplicati aggregando i valori (simile a GROUP BY + PIVOT in SQL)

5.2 Uso di Pivot

```python
# pivot richiede chiavi uniche
daily_store = df.groupby(['data', 'negozio'], as_index=False)['totale'].sum()
pivot_df = daily_store.pivot(index='data', columns='negozio', values='totale')
```

5.3 Uso di Pivot_table (Consigliato)

```python
# Tabella pivot base
pivot_df = pd.pivot_table(
    df,
    values='Vendite',
    index='Prodotto',
    columns='Città',
    aggfunc='mean'  # 'sum', 'count', 'len', np.mean, etc.
)

# Con valori mancanti riempiti
pivot_df = pd.pivot_table(
    df,
    values='Vendite',
    index='Prodotto',
    columns='Città',
    aggfunc='sum',
    fill_value=0  # Sostituisce NaN con 0
)

# Multi-index su righe
pivot_df = pd.pivot_table(
    df,
    index=['Negozio', 'Categoria'],
    columns='Prodotto',
    values='Vendite',
    aggfunc='sum',
    fill_value=0
)

# Multiple values
pivot_df = pd.pivot_table(
    df,
    index='Negozio',
    columns='Categoria',
    values=['Vendite', 'Quantita'],
    aggfunc={'Vendite': 'sum', 'Quantita': 'mean'}
)

# Marginali (somme per riga/colonna)
pivot_df = pd.pivot_table(
    df,
    index='Prodotto',
    columns='Città',
    values='Vendite',
    aggfunc='sum',
```

```
    margins=True,        # Aggiunge riga/colonna "All"
    margins_name='Totale'  # Nome delle righe/colonne marginali
)
```

5.4 Pivot_table Avanzato

```
# Multiple aggfunc
pivot_df = pd.pivot_table(
    df,
    index='Prodotto',
    columns='Città',
    values='Vendite',
    aggfunc=['sum', 'mean', 'count']
)

# Binning (crea categorie da valori continui)
df['categoria_prezzo'] = pd.cut(df['prezzo'], bins=[0, 50, 100, 200, 500], labels=['Basso',

pivot_df = pd.pivot_table(
    df,
    index='categoria_prezzo',
    columns='Città',
    values='Vendite',
    aggfunc='sum'
)

# Qcut (binning basato su percentili)
df['quantile_prezzo'] = pd.qcut(df['prezzo'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

pivot_df = pd.pivot_table(
    df,
    index='quantile_prezzo',
    columns='Città',
    values='Vendite',
    aggfunc='sum'
)
```

Parte VI: Ordinamento e Value Counts

6.1 Ordinamento

```
# Ordina per una colonna
df.sort_values('spesa')              # Ascendente (default)
df.sort_values('spesa', ascending=False)  # Discendente

# Ordina per multiple colonne
df.sort_values(['citta', 'eta'], ascending=[True, False])

# Ordina per indice
df.sort_index()

# Prime/N ultime righe dopo ordinamento
df.nlargest(5, 'spesa')   # 5 righe con spesa più alta
df.nsmallest(3, 'eta')    # 3 righe con eta più bassa
```

6.2 Value Counts

```
# Conteggio valori in una colonna
df['citta'].value_counts()

# Con percentuali
df['citta'].value_counts(normalize=True)

# Limita risultati
df['citta'].value_counts(head=3)

# Con categorie ordinate
df['citta'].value_counts(sort=True)  # Default: ordina per conteggio

# Per più colonne (Pandas >= 1.1.0)
df.value_counts(subset=['citta', 'categoria'])
```

Parte VII: Unione di DataFrame (Merge)

7.1 Tipi di Join

```
# LEFT JOIN (mantiene tutte le righe del sinistra)
pd.merge(df_vendite, df_costi, on='Prodotto', how='left')

# RIGHT JOIN (mantiene tutte le righe del destra)
pd.merge(df_vendite, df_costi, on='Prodotto', how='right')

# INNER JOIN (solo righe con corrispondenza in entrambi)
pd.merge(df_vendite, df_costi, on='Prodotto', how='inner')

# FULL OUTER JOIN (tutte le righe, con NaN dove non c'è corrispondenza)
pd.merge(df_vendite, df_costi, on='Prodotto', how='outer')
```

7.2 Merge Avanzato

```
# Merge su colonne con nomi diversi
pd.merge(df1, df2, left_on='nome_prodotto', right_on='prodotto')

# Merge su multiple colonne
pd.merge(df1, df2, on=['colonna1', 'colonna2'])

# Merge con indici
pd.merge(df1, df2, left_index=True, right_index=True)

# Aggiungere suffissi a colonne duplicate
pd.merge(df1, df2, on='id', suffixes=('_df1', '_df2'))
```

7.3 Concatenazione

```
# Concatena verticalmente (righe)
pd.concat([df1, df2, df3], axis=0, ignore_index=True)

# Concatena orizzontalmente (colonne)
pd.concat([df1, df2], axis=1)

# Concatena con ignore_index (riindice da 0)
result = pd.concat([df1, df2], ignore_index=True)
```

Parte VIII: Gestione del Tempo

8.1 Conversione a Datetime

```
# Conversione colonna a datetime
df['data'] = pd.to_datetime(df['data'])

# Specifica formato
df['data'] = pd.to_datetime(df['data'], format='%Y-%m-%d')

# Errori non validi diventano NaN
df['data'] = pd.to_datetime(df['data'], errors='coerce')

# Crea DatetimeIndex
df = df.set_index('data')

# Da stringhe multiple formati (Pandas prova a indovinare)
df['data'] = pd.to_datetime(df['data'])
```

8.2 Estrazione di Componenti Data/Ora

```
# Data
df['anno'] = df['data'].dt.year
df['mese'] = df['data'].dt.month
df['giorno'] = df['data'].dt.day
df['giorno_settimana'] = df['data'].dt.dayofweek  # 0=Lun, 6=Dom

# Ora
df['ora'] = df['data'].dt.hour
df['minuto'] = df['data'].dt.minute
df['secondo'] = df['data'].dt.second

# Settimane e mesi
df['settimana'] = df['data'].dt.isocalendar().week
df['trimestre'] = df['data'].dt.quarter

# Periodo (per raggruppamento)
df['mese_periodo'] = df['data'].dt.to_period('M')
df['anno_mese'] = df['data'].dt.to_period('Y-M')
```

8.3 Resample (Serie Temporali)

Il `resample()` funziona solo su DataFrame/Series con DatetimeIndex.

```python
# Imposta DatetimeIndex
df = df.set_index('data')

# Resample giornaliero (media)
df_daily = df.resample('D').mean()

# Resample mensile (somma)
df_monthly = df.resample('M').sum()

# Resample settimanale
df_weekly = df.resample('W').sum()

# Frequenze comuni:
# 'D' = giorno
# 'W' = settimana (domenica come fine settimana)
# 'M' = mese (fine mese)
# 'Q' = trimestre (fine trimestre)
# 'Y' o 'A' = anno (fine anno)
# 'H' = ora
# 'T' o 'min' = minuto
# 'S' = secondo

# Resample con multiple aggregazioni
df_monthly = df.resample('M').agg({
    'vendite': 'sum',
    'quantita': 'mean',
    'prezzo': 'max'
})
```

8.4 Rolling Windows (Finestre Scorrevoli)

```python
# Media mobile di 7 giorni
df['media_7g'] = df['valore'].rolling(window=7).mean()

# Deviazione standard mobile
df['std_7g'] = df['valore'].rolling(window=7).std()

# Finestra minima (almeno n valori validi)
df['media_7g_min3'] = df['valore'].rolling(window=7, min_periods=3).mean()

# Finestra esponenziale (pesa di più i valori recenti)
df['ewm'] = df['valore'].ewm(span=7).mean()

# Finestra su gruppi
df['media_per_negozio'] = df.groupby('negozio')['valore'].rolling(window=7).mean()
```

8.5 Shift e Pct_Change

```python
# Valore del periodo precedente
df['precedente'] = df['valore'].shift(1)

# Valore di n periodi precedenti
df['due_periodi_fa'] = df['valore'].shift(2)

# Variazione assoluta
df['diff'] = df['valore'] - df['valore'].shift(1)

# Variazione percentuale
df['pct_change'] = df['valore'].pct_change()
df['pct_change_100'] = df['valore'].pct_change() * 100

# Lags e leads multipli
df['lag1'] = df['valore'].shift(1)
df['lag2'] = df['valore'].shift(2)
df['lead1'] = df['valore'].shift(-1)
```

8.6 Timezone

```python
# Localizzare una datetime senza timezone
df['data'] = df['data'].dt.tz_localize('Europe/Rome')

# Convertire timezone
df['data'] = df['data'].dt.tz_convert('UTC')

# Convertire tutto il DataFrame a una timezone
df = df.tz_convert('UTC')
```

Parte IX: Indicizzazione Avanzata

9.1 MultiIndex

```python
# Crea MultiIndex da colonne
df_mi = df.set_index(['negozio', 'categoria', 'data'])

# Selezione con loc
df_mi.loc[('Genova', 'Tech')]  # Tutti i dati per Genova/Tech

# Slice su più livelli
df_mi.loc[('Genova', 'Tech', slice('2025-01-15', '2025-01-20'))]

# Cross-section (xs)
df_mi.xs('Genova', level='negozio')  # Solo Genova

# IndexSlice per slicing complesso
idx = pd.IndexSlice
df_mi.loc[idx['Genova':'Roma', 'Tech', :]]
```

9.2 Categorical Data

```python
# Converte colonna in categoria
df['citta'] = df['citta'].astype('category')

# Ordina categorie
df['citta'] = pd.Categorical(df['citta'],
                             categories=['Roma', 'Milano', 'Napoli'],
                             ordered=True)

# Aggiungi categorie
df['citta'] = df['citta'].cat.add_categories(['Torino'])

# Rimuovi categorie non usate
df['citta'] = df['citta'].cat.remove_unused_categories()
```

Parte X: Esercizi Pratici

10.1 Esercizio 1: Analisi Esplorativa

Obiettivo: Familiarizzare con le operazioni di base per l'esplorazione dei dati.

Passaggi chiave:

1. Generazione dataset con valori mancanti e duplicati
2. Visualizzazione con `head()`, `tail()`, `info()`, `describe()`
3. Rimozione duplicati con `drop_duplicates()`
4. Gestione missing values con `dropna()` e `fillna()`
5. Creazione colonna categoria con `apply()`
6. Salvataggio con `to_csv()`

10.2 Esercizio 2: Manipolazione e Aggregazione

Obiettivo: Manipolazione e aggregazione con groupby e pivot.

Passaggi chiave:

1. Creazione colonna derivata (Totale Vendite = Quantità × Prezzo)
2. Groupby per aggregazione: `groupby('Prodotto')['Vendite'].sum()`
3. Trovare massimo con `idxmax()`
4. Filtraggio con condizioni booleane
5. Ordinamento con `sort_values()`

6. Conteggio con `value_counts()`

10.3 Esercizio Telecomunicazioni (Churn Prediction)

Obiettivo: Pipeline completa di pulizia e preparazione dati.

Passaggi chiave:

1. Caricamento e esplorazione iniziale
2. Pulizia: rimozione duplicati, imputazione missing values
3. Correzione valori anomali con `clip()`
4. Feature engineering: `Costo_per_GB = Tariffa / Dati_Consumati`
5. Groupby per analisi per churn
6. Correlazioni con `corr()`
7. Conversione target categorico in numerico
8. Normalizzazione con `StandardScaler`
9. Split train/test manuale

Parte XI: Visualizzazione dei Dati

11.1 Introduzione a Matplotlib

Matplotlib è la libreria fondamentale per la visualizzazione in Python. Fornisce un'interfaccia simile a MATLAB per creare grafici statici, animati e interattivi.

Struttura base:

```python
import matplotlib.pyplot as plt

# Creare figura
plt.figure(figsize=(10, 6))

# Creare grafico
plt.plot(x, y)

# Aggiungere titoli e etichette
plt.title('Titolo del Grafico')
plt.xlabel('Asse X')
plt.ylabel('Asse Y')

# Mostrare grafico
plt.show()
```

11.2 Tipi di Grafici Base

Grafico a Linee:

```python
plt.plot(x, y)
plt.plot(x, y1, label='Serie 1', color='blue', linewidth=2)
plt.plot(x, y2, label='Serie 2', color='red', linestyle='--')
plt.legend()
plt.show()
```

Grafico a Barre:

```python
plt.bar(categorie, valori)
plt.bar(categorie, valori, color='skyblue', edgecolor='black')
plt.xticks(rotation=45)
plt.show()

# Barre orizzontali
plt.barh(categorie, valori)
```

Istogramma:

```
# Istogramma base
plt.hist(dati, bins=30, edgecolor='black')

# Istogramma con densità normalizzata
plt.hist(dati, bins=30, density=True, alpha=0.7)

# Istogramma cumulativo
plt.hist(dati, bins=30, cumulative=True)
```

Scatter Plot:

```
plt.scatter(x, y, alpha=0.5, s=50)  # s = dimensione punti

# Scatter con colori
plt.scatter(x, y, c=colore, alpha=0.5)

# Scatter con dimensione variabile
plt.scatter(x, y, s=dimensione, alpha=0.5)
```

11.3 Grafici Multipli in una Figura

```python
# Subplots (griglia di grafici)
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Grafico in specifica posizione
axes[0, 0].plot(x, y1)
axes[0, 0].set_title('Grafico 1')
axes[0, 1].plot(x, y2)
axes[0, 1].set_title('Grafico 2')
axes[1, 0].hist(dati)
axes[1, 0].set_title('Istogramma')
axes[1, 1].scatter(x, y)
axes[1, 1].set_title('Scatter')

plt.tight_layout()
plt.show()

# Subplots con layout specifico
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(x, y)
axes[0].set_title('Grafico 1')
axes[1].hist(dati)
axes[1].set_title('Istogramma')
plt.tight_layout()
plt.show()
```

11.4 Annotazioni e Elementi Aggiuntivi

```python
# Linea orizzontale
plt.axhline(y=media, color='red', linestyle='--', label='Media')

# Linea verticale
plt.axvline(x=valore, color='blue', linestyle='--')

# Area tra due curve
plt.fill_between(x, y1, y2, alpha=0.3, color='gray')

# Annotazione testo
plt.annotate('Massimo', xy=(x_max, y_max),
             xytext=(x_max+1, y_max+10),
             arrowprops=dict(arrowstyle='->', color='red'))

# Grid
plt.grid(True, alpha=0.3)
```

11.5 Seaborn: Visualizzazione Statistica

Seaborn è costruito sopra Matplotlib e offre un'interfaccia più alta per grafici statistici belli e informativi.

Caratteristiche principali:

- Stile predefinito più moderno
- Grafici statistici complessi con poche righe di codice
- Integrazione nativa con DataFrame pandas
- Supporto per dati "tidy" (lungi)

11.6 Grafici Seaborn

Bar Plot:

```
import seaborn as sns

# Bar plot con errore standard
sns.barplot(x='categoria', y='valore', data=df)

# Bar plot con media personalizzata
sns.barplot(x='giorno', y='total_bill', data=tips, estimator=sum)

# Ordine delle categorie
sns.barplot(x='categoria', y='valore', data=df,
            order=['A', 'B', 'C', 'D'])
```

Line Plot:

```
# Line plot base
sns.lineplot(x='time', y='valore', data=df)

# Multiple linee con hue
sns.lineplot(x='time', y='valore', hue='categoria', data=df)

# Linee con stile diverso
sns.lineplot(x='time', y='valore', hue='categoria',
             style='evento', data=df)
```

Histplot e KDE:

```
# Istogramma
sns.histplot(data=df, x='colonna', bins=30)

# Istogramma con densità (KDE)
sns.histplot(data=df, x='colonna', kde=True, bins=30)

# Istogramma con normalizzazione
sns.histplot(data=df, x='colonna', stat='density', bins=30)
sns.histplot(data=df, x='colonna', stat='percent', bins=30)

# Istogramma congiunto (jointplot)
sns.jointplot(data=df, x='colonna1', y='colonna2', kind='hist')

# Istogramma congiunto con KDE
sns.jointplot(data=df, x='colonna1', y='colonna2', kind='kde')
```

Box Plot e Violin Plot:

```
# Box plot
sns.boxplot(x='categoria', y='valore', data=df)

# Violin plot (combina box plot con KDE)
sns.violinplot(x='categoria', y='valore', data=df)

# Violin plot con punti dei dati
sns.violinplot(x='categoria', y='valore', data=df, inner='stick')
sns.violinplot(x='categoria', y='valore', data=df, inner='quartile')
sns.violinplot(x='categoria', y='valore', data=df, inner='points')
```

Heatmap:

```
# Matrice di correlazione
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)

# Heatmap con colori diversi
sns.heatmap(corr_matrix, annot=True, cmap='viridis',
            fmt='.2f', linewidths=0.5)
```

Pairplot:

```
# Matrice di scatter plot per tutte le coppie di colonne
sns.pairplot(df, hue='categoria')

# Solo colonne numeriche
sns.pairplot(df[['col1', 'col2', 'col3']])

# Pairplot con KDE sui diagonali
sns.pairplot(df, hue='categoria', diag_kind='kde')
```

11.7 Configurazione dello Stile

```
# Stili predefiniti
sns.set_style("whitegrid")   # white, dark, whitegrid, darkgrid, dark
sns.set_style("darkgrid")

# Tema colori
sns.set_palette("husl")      # husl, pastel, muted, bright, dark, colorblind
sns.set_palette("Set2")

# Dimensione figure
sns.set_context("notebook")  # paper, notebook, talk, poster
sns.set_context("talk", font_scale=1.5)

# Configurazione completa
sns.set_style("whitegrid")
sns.set_palette("colorblind")
sns.set_context("notebook", font_scale=1.2)
```

Parte XII: Normalizzazione e Standardizzazione

12.1 Perché Normalizzare?

La normalizzazione è essenziale quando:

- Le feature hanno scale molto diverse (es: età 0-100 vs reddito 0-100000)
- Si usano algoritmi sensibili alla scala (KNN, SVM, reti neurali, K-means)
- Si vuole dare lo stesso peso a tutte le feature

12.2 Min-Max Scaling

Formula:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Risultato: Tutti i valori sono nell'intervallo [0, 1]

Implementazione:

```
# Implementazione manuale
def min_max_scaling(serie):
    return (serie - serie.min()) / (serie.max() - serie.min())

# Con pandas
df['altezza_norm'] = (df['altezza'] - df['altezza'].min()) / \
                     (df['altezza'].max() - df['altezza'].min())

# Con sklearn
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df[['altezza', 'peso']])
```

12.3 Standard Scaling (Z-Score)

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Risultato: Media = 0, Deviazione Standard = 1

Implementazione:

```
# Implementazione manuale
def standard_scaling(serie):
    return (serie - serie.mean()) / serie.std()

# Con pandas
df['altezza_std'] = (df['altezza'] - df['altezza'].mean()) / df['altezza'].std()

# Con sklearn
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[['altezza', 'peso']])
df_scaled = pd.DataFrame(df_scaled, columns=df.columns, index=df.index)
```

12.4 Quando Usare Quale?

Metodo	Quando Usare	Vantaggi	Svantaggi
Min-Max	Quando i limiti sono noti/fissi	Interpretabilità [0,1]	Sensibile agli outlier
Standard	Quando la distribuzione è normale	Robusto per algoritmi ML	Valori possono essere negativi
Robust Scaling	Con molti outlier	Usa mediana e IQR	Meno comune

Parte XIII: Best Practices e Errori Comuni

13.1 Best Practices Generali

1. Importazione:

```
# Import standard
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configurazione display
pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 140)
pd.set_option('display.max_rows', 100)
```

2. Copia di DataFrame:

```python
# Sempre usare .copy() quando si crea un DataFrame derivato
df_new = df.copy()

# Evita il SettingWithCopyWarning
# SBAGLIATO:
df_subset = df[df['eta'] > 25]
df_subset['nuova_colonna'] = 10  # Warning!

# CORRETTO:
df_subset = df[df['eta'] > 25].copy()
df_subset['nuova_colonna'] = 10
```

3. Gestione dei tipi di dato:

```python
# Controlla e imposta tipi di dato appropriati
df['eta'] = df['eta'].astype(int)
df['data'] = pd.to_datetime(df['data'])
df['categoria'] = df['categoria'].astype('category')  # Per colonne con poche categorie

# Ottimizza memoria per categorie
df['citta'] = df['citta'].astype('category')
```

4. Catena di operazioni (method chaining):

```python
# Approccio fluido e leggibile
df_clean = (df
    .drop_duplicates()
    .dropna(subset=['eta', 'spesa'])
    .assign(spesa_iva=lambda x: x['spesa'] * 1.22)
    .assign(categoria_eta=lambda x: x['eta'].apply(classifica))
)
```

13.2 Errori Comuni da Evitare

1. Modificare DataFrame copiati:

```python
# SBAGLIATO: Crea un riferimento, non una copia
df_subset = df[df['eta'] > 25]
df_subset['nuova'] = 10  # SettingWithCopyWarning

# CORRETTO: Crea una copia esplicita
df_subset = df[df['eta'] > 25].copy()
df_subset['nuova'] = 10
```

2. Usare loop invece di operazioni vettorializzate:

```python
# SBAGLIATO: Lento
for i in range(len(df)):
    df.loc[i, 'doppio'] = df.loc[i, 'eta'] * 2

# CORRETTO: Vettoriale e veloce
df['doppio'] = df['eta'] * 2
```

3. Forgettare di specificare axis in drop:

```python
# SBAGLIATO: Rimuove righe invece di colonne
df.drop('colonna')  # Cerca riga con nome 'colonna'

# CORRETTO: Rimuove colonna
df.drop('colonna', axis=1)
# oppure
df.drop(columns=['colonna'])
```

4. Confondere loc e iloc:

```python
# SBAGLIATO: iloc usa posizioni, non etichette
df.iloc['colonna']  # Errore!

# CORRETTO:
df.loc[:, 'colonna']  # Per etichette
df.iloc[:, 0]  # Per posizioni
```

5. Non gestire i missing values:

```python
# Controlla sempre i missing values prima di procedere
print(df.isna().sum())
print(df.isna().mean() * 100)
```

6. Usare inplace=True senza assegnare:

```
# SBAGLIATO: inplace=True modifica in loco ma non restituisce
df.dropna(inplace=True)  # Funziona ma meno flessibile

# CORRETTO: Assegna il risultato (più chiaro)
df = df.dropna()
```

13.3 Takeaways per la Revisione

Concetti Fondamentali:

- **Series**: Colonna etichettata unidimensionale
- **DataFrame**: Tabella bidimensionale con colonne di tipi diversi
- **loc vs iloc**: etichette vs posizioni
- **NaN**: Rappresentazione di valori mancanti
- **groupby**: Raggruppa e aggrega
- **pivot_table**: Riassume dati in tabella incrociata

Operazioni Chiave:

- **Selezione**: `df[col]` , `df.loc[]` , `df.iloc[]` , `df.query()`
- **Filtraggio**: Maschere booleane, `isin()` , `between()`
- **Aggregazione**: `groupby().agg()` , `pivot_table()`
- **Trasformazione**: `transform()` , `apply()` , `map()`
- **Unione**: `merge()` , `concat()`
- **Tempo**: `to_datetime()` , `resample()` , `rolling()` , `shift()`

Best Practices:

- **Sempre**: `.copy()` quando si crea un DataFrame derivato
- **Sempre**: Controllare missing values prima di procedere
- **Preferire**: Operazioni vettorializzate invece di loop
- **Preferire**: Method chaining per codice più leggibile
- **Attenzione**: Differenza tra `loc` (inclusivo) e `iloc` (esclusivo)

Conclusione

Questa guida copre l'intero spettro di concetti di Pandas e visualizzazione trattati nei materiali del repository, dalle basi di Series e DataFrame fino alle tecniche avanzate di aggregazione, gestione

del tempo e creazione di visualizzazioni efficaci.

Punti chiave da ricordare:

1. **Comprendere la struttura dati**: Series (1D) e DataFrame (2D) sono i mattoni fondamentali
2. **Selezione corretta**: Distinguere tra `loc` (etichette) e `iloc` (posizioni)
3. **Gestione missing values**: Controllare sempre e scegliere strategia appropriata
4. **Operazioni vettorializzate**: Sempre preferire a loop espliciti per performance
5. **GroupBy e aggregazione**: Potenti per riassumere e analizzare dati
6. **Pivot_table**: Essenziale per analisi incrociate
7. **Gestione del tempo**: DatetimeIndex, resample, rolling windows
8. **Visualizzazione**: Matplotlib per controllo totale, Seaborn per bellezza e statistica

Percorso di apprendimento consigliato:

1. Fondamenti (Series, DataFrame, selezione)
2. Manipolazione (colonne, missing values, duplicati)
3. Aggregazione (groupby, pivot_table)
4. Unione (merge, concat)
5. Tempo (datetime, resample, rolling)
6. Visualizzazione (matplotlib, seaborn)
7. Best practices e ottimizzazione

Buono studio!